

Phase_2_Processing_Slovenia_SII_demo

November 4, 2025

1 Slovenia Solvency reports Table S.02.01.02; Phase 2 Processing

This script is a continuation of `Phase_1_Extraction_Slovenia_SII_demo`. It shows the second Phase of this process. The processing phase applies different transformations to the raw OCR data from the previous phase. Examples of transformations are changing separators, removing trailing zeros, remove dolar symbols etc.

1.1 Companies in scope

For the year 2024, the companies in scope are the following:

- Zavarovalnica Triglav d.d.
- Generali zavarovalnica d.d.
- Prva osebna zavarovalnica, d.d.
- Modra zavarovalnica d.d.
- Vzajemna d.d.
- Grawe zavarovalnica d.d.
- Zavarovalnica Sava, d.d.

1.2 Description of the process

The process of extraction is performed in 5 phases:

1.2.1 Phase 0: Find the reports and identify the relevant tables.

- 1) Identify the new SFCR report and save it into the folder Input.
- 2) Identify the pages where the tables of interest are.
- 3) Compile the map of the company run in the `master_list.csv`.

1.2.2 Phase 1: Run the Extraction script.

The script performs the following steps (with slight modifications depending on the table format):

- 1) Save the page with the table into a separate folder `Single_pdf`.
- 2) Use either a Python package or specialized LLM to create a digital equivalent of the table.
- 3) Fix the systemic errors that prevent the table from being saved as DataFrame.
- 4) Save the DataFrame into the Output folder.

1.2.3 Phase 2: Run the Processing script (this script).

The script applies fixes to the DataFrame to make the numbers closer to the reported numbers. It joins all the tables into a single dataset and saves it into the `Dirty_Combined` folder.

1.2.4 Phase 3: Run the Cross-Validation script.

Applies a series of tests that check for the internal consistency between the numbers. Flags potential errors. After the individual fixes are applied, it saves the table into the Cleaner_Combined folder.

1.2.5 Phase 4: Final modifications to the table and a manual inspection.

1.3 Python packages

```
[140]: import pandas as pd
import numpy as np
```

1.4 Functions

```
[141]: def remove_trailing_zeros(data: pd.DataFrame, col: str, n: int) -> pd.DataFrame:
    """
    Removes exactly n trailing zeros from numbers in a column, if present.

    Args:
        data (pd.DataFrame): Input dataframe.
        col (str): Column name to process.
        n (int): Number of trailing zeros to remove.

    Returns:
        pd.DataFrame: DataFrame with modified column.
    """
    def clean_value(x):
        if pd.isna(x):
            return x
        try:
            # Convert to int first (in case it's float or string with decimals)
            x_int = int(float(x))
            str_x = str(x_int)
            zeros = "0" * n
            if str_x.endswith(zeros):
                return int(str_x[:-n]) # remove trailing zeros
            return x_int
        except ValueError:
            return x # return unchanged if not a valid number

    data[col] = data[col].apply(clean_value)
    return data
```

```
[142]: def adjust_decimals(data: pd.DataFrame, col: str = "C0010") -> pd.DataFrame:
    """
    Adjust values in a dataframe column:
    - If a value has non-zero decimal digits, multiply it by 1000.
    - Otherwise, leave it unchanged.
```

```

Parameters
-----
data : pd.DataFrame
    Input dataframe with numeric values.
col : str
    Column name to check (default "C0010").

Returns
-----
pd.DataFrame
    Updated dataframe with adjusted values.
"""
data = data.copy()
decimals = data[col] % 1
data[col] = np.where(decimals != 0, data[col] * 1000, data[col])
return data

```

```

[143]: def remove_space(data: pd.DataFrame, col: str = "C0010") -> pd.DataFrame:
        """
        Remove every space in a column of strings in a DataFrame.
        """
        data[col] = (
            data[col]
            .str.replace(" ", "", regex=False)    # remove thousand separators
        )
        return data

```

```

[144]: def remove_dot_sep(data: pd.DataFrame, col: str = "C0010") -> pd.DataFrame:
        """
        Remove every dot in a column of strings in a DataFrame.
        """
        data[col] = (
            data[col]
            .str.replace(".", "", regex=False)
        )
        return data

```

```

[145]: def as_float(data: pd.DataFrame, col: str = "C0010") -> pd.DataFrame:
        """
        Convert a column of strings in a DataFrame into floating numbers.
        """
        data[col] = (
            data[col]
            .astype(float)
        )
        return data

```

```
[146]: def as_string(data: pd.DataFrame, col: str = "C0010") -> pd.DataFrame:
        """
        Convert a column of strings in a DataFrame into floating numbers.
        """
        data[col] = (
            data[col]
            .astype(str)
        )
        return data
```

```
[147]: def remove_comma_sep(data: pd.DataFrame, col: str = "C0010") -> pd.DataFrame:
        """
        Remove every comma in a column of strings in a DataFrame.
        """
        data[col] = (
            data[col]
            .str.replace(",", "", regex=False)
        )
        return data
```

```
[148]: def remove_dolar_sep(data: pd.DataFrame, col: str = "C0010") -> pd.DataFrame:
        """
        Remove every dolar sign in a column of strings in a DataFrame.
        """
        data[col] = (
            data[col]
            .str.replace("$", "", regex=False)
        )
        return data
```

```
[149]: def replace_coma_with_dot(data: pd.DataFrame, col: str = "C0010") -> pd.
        DataFrame:
        """
        Replace every comma in a column of strings of a DataFrame with a dot.
        """
        data[col] = (
            data[col]
            .str.replace(",", ".", regex=False)
        )
        return data
```

```
[150]: def fix_dot_numbers(data: pd.DataFrame, col: str) -> pd.DataFrame:
        """
        Fix numbers in a column where dots are used as thousand separators
        but keep values like '0.000' unchanged.
```

```

Args:
    data (pd.DataFrame): Input dataframe.
    col (str): Column name to fix.

Returns:
    pd.DataFrame: DataFrame with the cleaned column.
"""

def clean_value(x):
    if pd.isna(x): # Handle NaN
        return x
    x_str = str(x)
    before, _, after = x_str.partition(".")
    if before == "0" and after == "000":
        return x_str # keep '0.000'
    return x_str.replace(".", "") # remove dots otherwise

data[col] = data[col].apply(clean_value)
return data

```

```

[151]: def replace_dash_with_zero(data: pd.DataFrame, col: str) -> pd.DataFrame:
        """
        Replace cells containing only '-' with 0 in the specified column.

        Parameters
        -----
        data : pd.DataFrame
            Input dataframe.
        col : str
            Column name where replacement should happen.

        Returns
        -----
        pd.DataFrame
            DataFrame with '-' replaced by 0 in the specified column.
        """
        data = data.copy()
        data[col] = data[col].apply(lambda x: 0 if str(x).strip() == "-" else x)
        data[col] = data[col].apply(lambda x: 0 if str(x).strip() == "-" else x)
        return data

```

```

[152]: def remove_bracket_instead_negative(data: pd.DataFrame, column: str) -> pd.
↳ DataFrame:
        """
        Converts numbers stored as strings with brackets into negative numbers.
        Example: "(123)" -> "-123"

```

```

Parameters:
    data (pd.DataFrame): Input DataFrame
    column (str): Column name where conversion should be applied

Returns:
    pd.DataFrame: Updated DataFrame with cleaned column
    """
    data = data.copy()

    def convert(value):
        if isinstance(value, str) and value.startswith("(") and value.
↪endswith(")"):
            return "-" + value[1:-1] # remove brackets, prepend "-"
        return value

    data[column] = data[column].apply(convert)
    return data

```

```

[153]: def remove_trailing_dot_zero(df: pd.DataFrame, column: str) -> pd.DataFrame:
    """
    Removes the trailing '.0' from string numbers in a specified column of a
↪DataFrame.

    Parameters
    -----
    df : pd.DataFrame
        Input DataFrame.
    column : str
        Name of the column containing string numbers to be modified.

    Returns
    -----
    pd.DataFrame
        DataFrame with the specified column modified - values ending with '.0'
↪have it removed.
    """
    df = df.copy()
    df[column] = df[column].astype(str).str.replace(r'\.0$', '', regex=True)
    return df

```

```

[154]: def initial_read(unique_id: str, list: pd.DataFrame) -> pd.DataFrame:
    """
    Find the location of the table and load it.
    """
    path = list.loc[unique_id, "output_final_path"]

    return pd.read_csv(path, index_col = 0).fillna(0)

```

```
[155]: def shorten_index(data: pd.DataFrame) -> pd.DataFrame:
        """
        Trims the index values so they contain only the first 5 characters.

        Example:
            'R0510 2.337.991' -> 'R0510'
        """
        data = data.copy()
        data.index = data.index.astype(str).str[:5]
        return data
```

```
[156]: def convert_index_to_R(data: pd.DataFrame) -> pd.DataFrame:
        """
        Converts the dataframe index by replacing the first digit with 'R'
        and formatting the rest as string without decimals.

        Example:
            80000.0 -> "R0000"
            80500.0 -> "R0500"
        """
        data = data.copy()
        data.index = (
            data.index.astype(int).astype(str).str[1:].str.zfill(4).map(lambda x:
↪ "R" + x)
        )
        return data
```

```
[157]: def normal_remove_dot(table):
        table = table[table.index.notnull()]
        table = table[~table.index.isna()]
        table = as_string(table, "C0010")
        table = remove_space(table, "C0010")
        table = remove_dot_sep(table, "C0010")
        table = remove_dolar_sep(table, "C0010")
        table = as_float(table, "C0010")
        table = table.fillna(0)
        return table
```

```
[158]: def normal_with_coma_sep(table):
        table = table[table.index.notnull()]
        table = table[~table.index.isna()]
        table = as_string(table, "C0010")
        table = remove_dot_sep(table, "C0010")
        table = replace_coma_with_dot(table, "C0010")
        table = remove_dolar_sep(table, "C0010")
        table = remove_space(table, "C0010")
        table = as_float(table, "C0010")
```

```
table = table.fillna(0)
return table
```

```
[159]: def normal_with_trailing_zeros(table):
        table = table[table.index.notnull()]
        table = table[~table.index.isna()]
        table = as_string(table, "C0010")
        table = remove_trailing_dot_zero(table, "C0010")
        table = remove_space(table, "C0010")
        table = remove_dot_sep(table, "C0010")
        table = remove_dolar_sep(table, "C0010")
        table = as_float(table, "C0010")
        table = table.fillna(0)
        return table
```

2 Master list of companies

```
[160]: master_list = pd.read_csv("master_list.csv", header=0, index_col=0)
```

```
[161]: display(master_list)
```

	company	document_name \
VZAJEMNA		
GENERALI_02_1	GENERALI	Input\Porocilo-o-solventnosti-in-financnem-pol...
GENERALI_02_2	GENERALI	Input\Porocilo-o-solventnosti-in-financnem-pol...
TRIGLAV_02_1	TRIGLAV	Input\Poročilo+o+solventnosti+in+finančnem+pol...
TRIGLAV_02_2	TRIGLAV	Input\Poročilo+o+solventnosti+in+finančnem+pol...
PRVA_02_1	PRVA	Input\Prva-osebna-zavarovalnica-PSFP_2024-2025...
PRVA_02_2	PRVA	Input\Prva-osebna-zavarovalnica-PSFP_2024-2025...
MODRA_02_1	MODRA	Input\Revidirano-SFCR-porocilo-2024.pdf
MODRA_02_2	MODRA	Input\Revidirano-SFCR-porocilo-2024.pdf
MODRA_02_3	MODRA	Input\Revidirano-SFCR-porocilo-2024.pdf
VZAJEMNA_02_1	VZAJEMNA	Input\Vzajemna_2024.pdf
VZAJEMNA_02_2	VZAJEMNA	Input\Vzajemna_2024.pdf
GRAWE_02_1	GRAWE	Input\Grawe_2024.pdf
GRAWE_02_2	GRAWE	Input\Grawe_2024.pdf
SAVA_02_1	SAVA	Input\Sava-za-leto-2024.pdf
SAVA_02_2	SAVA	Input\Sava-za-leto-2024.pdf

	table_name	page_number \
VZAJEMNA		
GENERALI_02_1	S.02.01.02_A	83
GENERALI_02_2	S.02.01.02_L	84
TRIGLAV_02_1	S.02.01.02_A	110
TRIGLAV_02_2	S.02.01.02_L	111
PRVA_02_1	S.02.01.02_A	63
PRVA_02_2	S.02.01.02_L	64

MODRA_02_1	S.02.01.02_A	83
MODRA_02_2	S.02.01.02_L	84
MODRA_02_3	S.02.01.02_L2	85
VZAJEMNA_02_1	S.02.01.02_A	72
VZAJEMNA_02_2	S.02.01.02_L	73
GRAWE_02_1	S.02.01.02_A	112
GRAWE_02_2	S.02.01.02_L	113
SAVA_02_1	S.02.01.02_A	104
SAVA_02_2	S.02.01.02_L	105

output_pdf_path \

VZAJEMNA

GENERALI_02_1	Single_pdf/GENERALI_S02_01_02_1_2024.pdf
GENERALI_02_2	Single_pdf/GENERALI_S02_01_02_2_2024.pdf
TRIGLAV_02_1	Single_pdf/TRIGLAV_S02_01_02_1_2024.pdf
TRIGLAV_02_2	Single_pdf/TRIGLAV_S02_01_02_2_2024.pdf
PRVA_02_1	Single_pdf/PRVA_S02_01_02_1_2024.pdf
PRVA_02_2	Single_pdf/PRVA_S02_01_02_2_2024.pdf
MODRA_02_1	Single_pdf/MODRA_S02_01_02_1_2024.pdf
MODRA_02_2	Single_pdf/MODRA_S02_01_02_2_2024.pdf
MODRA_02_3	Single_pdf/MODRA_S02_01_02_3_2024.pdf
VZAJEMNA_02_1	Single_pdf/VZAJEMNA_S02_01_02_1_2024.pdf
VZAJEMNA_02_2	Single_pdf/VZAJEMNA_S02_01_02_2_2024.pdf
GRAWE_02_1	Single_pdf/GRAWE_S02_01_02_1_2024.pdf
GRAWE_02_2	Single_pdf/GRAWE_S02_01_02_2_2024.pdf
SAVA_02_1	Single_pdf/SAVA_S02_01_02_1_2024.pdf
SAVA_02_2	Single_pdf/SAVA_S02_01_02_2_2024.pdf

output_final_path \

VZAJEMNA

GENERALI_02_1	Output/GENERALI_S02_01_02_1_2024.csv
GENERALI_02_2	Output/GENERALI_S02_01_02_2_2024.csv
TRIGLAV_02_1	Output/TRIGLAV_S02_01_02_1_2024.csv
TRIGLAV_02_2	Output/TRIGLAV_S02_01_02_2_2024.csv
PRVA_02_1	Output/PRVA_S02_01_02_1_2024.csv
PRVA_02_2	Output/PRVA_S02_01_02_2_2024.csv
MODRA_02_1	Output/MODRA_S02_01_02_1_2024.csv
MODRA_02_2	Output/MODRA_S02_01_02_2_2024.csv
MODRA_02_3	Output/MODRA_S02_01_02_3_2024.csv
VZAJEMNA_02_1	Output/VZAJEMNA_S02_01_02_1_2024.csv
VZAJEMNA_02_2	Output/VZAJEMNA_S02_01_02_2_2024.csv
GRAWE_02_1	Output/GRAWE_S02_01_02_1_2024.csv
GRAWE_02_2	Output/GRAWE_S02_01_02_2_2024.csv
SAVA_02_1	Output/SAVA_S02_01_02_1_2024.csv
SAVA_02_2	Output/SAVA_S02_01_02_2_2024.csv

codes_path leto

VZAJEMNA

GENERALI_02_1	Codes/Codes_S02_A.csv	2024
GENERALI_02_2	Codes/Codes_S02_L.csv	2024
TRIGLAV_02_1	Codes/Codes_S02_A_Triglav.csv	2024
TRIGLAV_02_2	Codes/Codes_S02_L_Triglav.csv	2024
PRVA_02_1	NaN	2024
PRVA_02_2	NaN	2024
MODRA_02_1	NaN	2024
MODRA_02_2	NaN	2024
MODRA_02_3	NaN	2024
VZAJEMNA_02_1	NaN	2024
VZAJEMNA_02_2	NaN	2024
GRAWE_02_1	NaN	2024
GRAWE_02_2	NaN	2024
SAVA_02_1	NaN	2024
SAVA_02_2	NaN	2024

3 Code

3.1 Generali Slovenia

S.01.02.01 1

```
[162]: table_1 = initial_read("GENERALI_02_1", master_list)
```

```
[163]: table_1 = normal_remove_dot(table_1)
```

S.02.01.02 2

```
[164]: table_2 = initial_read("GENERALI_02_2", master_list)
```

```
[165]: table_2 = normal_with_trailing_zeros(table_2)
```

Skupaj

```
[166]: table = pd.concat([table_1, table_2])
table.columns = pd.MultiIndex.from_product([["GENERALI"], ["NAME", "COO10"]])
```

```
[167]: general_i02 = table
```

```
[168]: del table, table_1, table_2
```

3.2 TRIGLAV

S.02.01.02 1

```
[169]: table_1 = initial_read("TRIGLAV_02_1", master_list)
```

```
[170]: table_1 = normal_with_coma_sep(table_1)
```

S.02.01.02 2

```
[171]: table_2 = initial_read("TRIGLAV_02_2", master_list)
```

```
[172]: table_2 = normal_with_trailing_zeros(table_2)
```

Skupaj

```
[173]: table = pd.concat([table_1, table_2])
```

```
[174]: table.columns = pd.MultiIndex.from_product([["TRIGLAV"], ["NAME", "C0010"]])
```

```
[175]: triglav_02 = table
```

```
[176]: del table_1, table_2
```

3.3 Prva osebna zavarovalnica, d.d.

S.02.01.02 1

```
[177]: table_1 = initial_read("PRVA_02_1", master_list)
```

```
[178]: table_1 = normal_with_coma_sep(table_1)
```

S.02.01.02 2

```
[179]: table_2 = initial_read("PRVA_02_2", master_list)
```

```
[180]: table_2 = normal_with_trailing_zeros(table_2)
```

Skupaj

```
[181]: table = pd.concat([table_1, table_2])  
table.columns = pd.MultiIndex.from_product([["PRVA"], ["NAME", "C0010"]])
```

```
[182]: prva_02 = table
```

```
[183]: del table, table_1, table_2
```

3.4 Modra zavarovalnica d.d.

S.02.01.02 1

```
[184]: table_1 = initial_read("MODRA_02_1", master_list)
```

```
[185]: table_1 = as_float(table_1, "C0010")  
table_1 = table_1.fillna(0)
```

S.02.01.02 2

```
[186]: table_2 = initial_read("MODRA_02_2", master_list)
```

```
[187]: table_2 = table_2.drop(index=(":--"))  
table_2 = table_2.drop(index=("C0010"))
```

```
[188]: table_2 = normal_with_coma_sep(table_2)
```

S.02.01.02 3

```
[189]: table_3 = initial_read("MODRA_02_3", master_list)
```

```
[190]: table_3 = normal_with_coma_sep(table_3)
```

Skupaj

```
[191]: table = pd.concat([table_1, table_2, table_3])  
table.columns = pd.MultiIndex.from_product([["MODRA"], ["NAME", "COO10"]])
```

```
[192]: modra_02 = table
```

```
[193]: del table, table_1, table_2, table_3
```

3.5 Vzajemna d.d.

S.02.01.02 1

```
[194]: table_1 = initial_read("VZAJEMNA_02_1", master_list)
```

```
[195]: table_1 = normal_remove_dot(table_1)
```

S.02.01.02 2

```
[196]: table_2 = initial_read("VZAJEMNA_02_2", master_list)
```

```
[197]: table_2 = normal_with_trailing_zeros(table_2)
```

Skupaj

```
[198]: table = pd.concat([table_1, table_2])  
table.columns = pd.MultiIndex.from_product([["VZAJEMNA"], ["NAME", "COO10"]])
```

```
[199]: vzajemna_02 = table
```

```
[200]: del table, table_1, table_2
```

3.6 Grawe zavarovalnica d.d.

S.02.01.02 1

```
[201]: table_1 = initial_read("GRAWE_02_1", master_list)
```

```
[202]: table_1 = normal_with_coma_sep(table_1)
```

```
[203]: table_1 = table_1[~table_1.index.isna()]
```

S.02.01.02 2

```
[204]: table_2 = initial_read("GRAWE_02_2", master_list)
```

```
[205]: table_2 = normal_with_trailing_zeros(table_2)
```

Skupaj

```
[206]: table = pd.concat([table_1, table_2])
       table.columns = pd.MultiIndex.from_product([["GRAWE"], ["NAME", "CO010"]])
```

```
[207]: grawe_02 = table
```

```
[208]: del table, table_1, table_2
```

3.7 Zavarovalnica Sava, d.d.

S.02.01.02 1

```
[209]: table_1 = initial_read("SAVA_02_1", master_list)
```

```
[210]: table_1 = normal_with_coma_sep(table_1)
```

S.02.01.02 2

```
[211]: table_2 = initial_read("SAVA_02_2", master_list)
```

```
[212]: table_2 = normal_with_coma_sep(table_2)
```

Skupaj

```
[213]: table = pd.concat([table_1, table_2])
       table.columns = pd.MultiIndex.from_product([["SAVA"], ["NAME", "CO010"]])
```

```
[214]: sava_02 = table
```

```
[215]: del table, table_1, table_2
```

4 Dirty master table

```
[216]: master = pd.read_csv("Input/S02_01_02_Master_Table.csv", index_col = 0).
       ↪ fillna(0)
       master.columns = pd.MultiIndex.from_product([["MASTER"], ["NAME"]])
```

```
[217]: generali_02 = generali_02.drop(columns=("GENERALI", "NAME"))
       triglav_02 = triglav_02.drop(columns=("TRIGLAV", "NAME"))
       prva_02 = prva_02.drop(columns=("PRVA", "NAME"))
       modra_02 = modra_02.drop(columns=("MODRA", "NAME"))
       vzajemna_02 = vzajemna_02.drop(columns=("VZAJEMNA", "NAME"))
       grawe_02 = grawe_02.drop(columns=("GRAWE", "NAME"))
       sava_02 = sava_02.drop(columns=("SAVA", "NAME"))
```

```
[218]: final = master.join(modra_02, how='left').join(triglav_02, how='left').
       ↪ join(prva_02, how='left').join(generali_02, how='left').
       ↪ join(vzajemna_02, how='left').join(grawe_02, how='left').
       ↪ join(sava_02, how='left')
```

```
[219]: final = final[~final.index.duplicated(keep="first")]  
       final = final.fillna(0)
```

```
[220]: final.to_csv("Dirty_Combined/Table_slo_S02.csv")
```